ChatGPT

Low-Cost Chinese Translation Resources for Level 1 Word Packages

Executive summary

For your specific schema, `translation_zh` and `definition_zh` should not be treated as the same problem. `translation_zh` is mostly a sentence-translation task, so low-cost MT APIs are a good fit. `definition_zh` is a lexicography task: you want short, learner-friendly Chinese glosses that match CEFR level, part of speech, and classroom tone. In practice, the strongest low-cost setup is a hybrid one: use a cheap MT system for sentence drafts, use lexicon resources such as CC-CEDICT or Wiktionary only as candidate/validation sources, and then normalize the output with a tiny post-editing step, either with rules or a very cheap LLM. That is safer, cheaper, and easier to QA than trying to make one source do everything. [1]

If I were implementing a Level 1 pilot today for 20–80 words, I would prioritize **Azure Translator F0** as the primary MT source, because Microsoft's free tier includes **2 million characters per month**, the API is simple JSON, Chinese Simplified and Traditional are both explicitly supported, request limits are generous, and Microsoft says attribution is **not required** for Azure Translator text and speech translation. That combination is unusually favorable for a commercial learning app pilot. Google Cloud Translation is also strong, especially if you already use Google Cloud, but Google's consumer translator is not the product you should integrate against; the official programmatic option is Cloud Translation, and raw output display normally requires Google attribution unless you post-edit the text. [2]

For fully free or offline workflows, **LibreTranslate** and **Argos Translate** are viable, but they are best used as baseline translation engines or privacy-preserving fallbacks, not as your only production-quality source for learner-facing Chinese. Their biggest advantages are zero marginal API cost, offline capability, and easy self-hosting. Their biggest weakness is quality consistency on short, low-context educational strings, where cloud MT systems with better handling of context, glossaries, and domain adaptation usually perform better. [3]

The two open lexicon resources you asked to include, **CC-CEDICT** and **Wiktionary**, are not good primary engines for `translation_zh`, but they are valuable for `definition_zh` candidate generation, synonym checking, and QA. The important caution is licensing: both are share-alike resources, and CC-CEDICT's own official pages are inconsistent about whether the current license is **CC BY-SA 3.0** or **CC BY-SA 4.0**. For a commercial app, that means you should treat them as **candidate sources and validators**, keep an audit trail, and avoid copying long verbatim definitions/examples into proprietary production rows without legal review. That is a practical product-risk recommendation, not legal advice. [4]

What matters most for your use case

Your app does not just need “a Chinese translation.” It needs Chinese text that is short, stable, learner-friendly, and easy to turn into questions. That creates three practical requirements.

First, the MT system should be good at **very short inputs**. DeepL explicitly exposes a `context` parameter for short, low-context strings such as product names, headlines, and UI elements, and those context characters are not billed. Google Cloud Translation offers glossaries and adaptive translation. These features matter because simple dictionary-style definitions such as “the physical or mental power to do something” are exactly the sort of short text where raw MT can be literal but pedagogically clumsy. [5]

Second, license safety matters more for `definition_zh` than for `translation_zh`. Machine-translation APIs are generally vendor SaaS services. By contrast, CC-CEDICT and Wiktionary are explicitly share-alike resources. Wiktionary states that original texts are dual-licensed under **CC BY-SA 4.0** and the **GFDL**, and the Wiktionary parsing page warns that the data is useful but not computer-friendly and that parsing is difficult. So if your goal is a clean Supabase pipeline, lexicon resources are better treated as structured evidence and QA material, not as unfiltered production text. [6]

Third, operational simplicity matters. Azure’s REST body is just a JSON array of `{ "text": ... }` objects. LibreTranslate accepts either a string or an array for `q`. DeepL accepts up to **50 texts per request**. Google Cloud Basic supports list input and Advanced supports asynchronous batch translation of large files. OpenAI’s Batch API allows **50,000 requests per file** at a **50% discount** versus synchronous calls, which makes it attractive as a post-editor if you want to rewrite only the rows that look awkward after MT. [7]

Comparison of the top options

The table below is implementation-focused. “Quality for short sentences” is a practical fit judgment for your app, not a formal benchmark score.

Name	Cost estimate	License	Quality for short sentences	API?	Batching	Notes
Azure Translator	**$0** up to **2M chars/ month** on F0; then **$10 per 1M chars** on S1 standard translation	Vendor service terms	High	Y	Up to **1,000** array elements; **50,000** chars/ request	Best free-tier value for pilot; supports `zh-Hans` and `zh-Hant`; attribution not required. [8]

Name	Cost estimate	License	Quality for short sentences	API?	Batching	Notes
Google Cloud Translation	**$0** for first **500k chars/ month**; then paid Cloud pricing	Vendor service terms	High	Y	Arrays for small requests; async batch for large jobs	Official developer option on Google Cloud; supports `zh-CN` / `zh` and `zh-TW`; raw output usually needs Google attribution, but post-edited output does not. [9]
DeepL API	Current public docs show plan transition; legacy docs show **500k free** in API Free and **1M included** in Growth	Vendor service terms	High	Y	Up to **50 texts/ request**	Strong short-text features via `context`; supports Simplified and Traditional Chinese as target variants; public plan naming/ pricing is currently in transition, so verify checkout terms. [10]

Name	Cost estimate	License	Quality for short sentences	API?	Batching	Notes
LibreTranslate	Free if self-hosted; hosted instances may require API keys/ pricing	**AGPL-3.0** for server code	Medium	Y	`q` can be string or array	Good open-source baseline; self-hosting avoids per-call cost but quality is below major cloud MT for learner-facing polish. [11]
Argos Translate	Free local/ offline	**MIT** for code	Medium to lower-medium	N	Script-level batching	Best offline core engine; downloadable models; can be wrapped behind LibreTranslate if you want HTTP access. [12]
CC-CEDICT	Free download	**CC BY-SA**, but official pages conflict between **3.0** and **4.0**	Low for sentences, useful for glosses	N	Offline file processing	Excellent as a lexical candidate/ validator; poor as a sentence translator; share-alike requires audit discipline. [13]

Name	Cost estimate	License	Quality for short sentences	API?	Batching	Notes
Wiktionary dumps and Action API	Free	**CC BY-SA 4.0 + GFDL**	Low to medium for gloss mining	Y, but not clean translation JSON	Dumps/ XML/SQL; API/dump processing	Valuable for candidate glosses, variants, and cross-checking; official docs explicitly say parsing is hard. [14]
LLM post-editing layer	OpenAI: pennies at small scale; local open models: near-zero marginal cost	OpenAI service terms or model-specific open licenses	High for output normalization	Y/N	OpenAI Batch or local script batching	Best used after MT, not instead of MT. OpenAI supports structured outputs and Batch; Qwen2.5 is Apache 2.0, multilingual, and can run behind an OpenAI-compatible local server. [15]

Resource-by-resource evaluation

Azure Translator is the most practical first choice for your pilot. The free tier is large enough that a Level 1 pilot will almost certainly cost nothing, and Microsoft explicitly recommends starting on F0 while learning the service. The request format is simple, Chinese Simplified and Traditional are both supported, and the service limits are comfortable for CSV-oriented processing: **50,000 characters per request**, up to **1,000 array elements**, and an hourly quota model documented for F0. Microsoft's FAQ also says attribution is not required, which removes an important product-design headache. The downside is that Azure is still a general MT system, so for `definition_zh` you should expect some literal phrasing unless you add a rewrite step. Recommended usage pattern: **Azure for raw sentence translations and raw gloss translation, then a compression/rewrite pass for `definition_zh`, with human spot-checking on pilot rows**. [16]

Google Cloud Translation is strong if you already have Google Cloud infrastructure or want glossaries and adaptive behavior in the same ecosystem. Google documents both simple text translation and asynchronous batch translation, and the product pages state that the first **500,000 characters per month** are free. Language support explicitly includes Simplified and Traditional Chinese. The biggest catch is product/legal UX: Google requires attribution when you display raw Cloud Translation results, although Google's FAQ says attribution is not required after post-editing. For your app, that means Google becomes much more attractive if your pipeline rewrites the raw MT into learner-friendly final copy before it reaches users. Recommended usage pattern: **Google raw MT + glossary if needed + post-edit before publishing**. [17]

DeepL API remains attractive for short educational strings because its API explicitly supports a non-billed `context` field and can process multiple texts in one request. It also exposes Simplified and Traditional Chinese as target variants. The practical problem is commercial clarity: the current official DeepL support materials show a transition in plan naming, with older `API Free` / `API Pro` language still visible in some docs and newer `API Developer` / `API Growth` language in others. The free/developer side also has an important data point in the current terms: DeepL's free developer offering reserves the right to store content perpetually, while the paid Growth terms describe temporary technical storage. For a classroom app, that makes DeepL more suitable as a paid option than a "totally free and legally simple" option. Recommended usage pattern: **use only if you want the quality/context feature set and are comfortable verifying current plan/data terms at purchase time**. [18]

LibreTranslate is the most practical open-source HTTP API option. Its docs are clean, the API is JSON, self-hosted instances can run without API keys, the server code is AGPL-3.0, and the `/translate` endpoint accepts either a string or an array for `q`. It also supports alternative translations and returns `429` for rate limiting. Language support includes `zh` and `zt` model pairs through the Argos ecosystem. The trade-off is that you are now the operator: quality, latency, uptime, and model freshness become your problem if you self-host. Recommended usage pattern: **use it as a zero-marginal-cost fallback or for offline/private translation, not as the only source for polished learner text**. [19]

Argos Translate is the best purely offline building block. The project is MIT-licensed, runs as a Python library/CLI/GUI, supports downloadable translation packages, and can pivot through intermediate languages when a direct pair is missing. It can also sit behind LibreTranslate if you want a local API. Its main weakness is quality stability on small educational prompts, especially when you care about subtle distinctions between "meaning," "gloss," and "natural classroom Chinese." Recommended usage pattern: **offline baseline or privacy fallback, plus a better post-editor**. [12]

CC-CEDICT is valuable, but not as a sentence translator. It is a Chinese-English dictionary resource meant for downloadable use, and official pages point you to downloadable releases. The important implementation point is legal: CC-CEDICT's official site currently has an inconsistency, with one page saying **CC BY-SA 3.0** and the download page saying **CC BY-SA 4.0**. That does not make it unusable, but it does make it something you should isolate in your pipeline. Use it to suggest short glosses, validate headword-to-Chinese mappings, and catch glaring MT errors. Do not make it the sole direct source of final Chinese strings unless you are comfortable with share-alike obligations and careful attribution handling. Recommended usage pattern: **candidate source and validator, not final production text**. [13]

Wiktionary is broader and messier. It is undeniably useful, especially when you want alternate senses, usage notes, or translation candidates for individual words, but the official Wiktionary parsing

documentation explicitly says the data is unfriendly for computer parsing and that “that day is not yet close” for a simple structured data API. The dumps are available, and the MediaWiki Action API can retrieve revisions/content, but you still need a parser and a quality layer. Since the text is share-alike, the same “use as evidence and candidate source, not verbatim production copy” logic applies here too. Recommended usage pattern: **backup candidate mining and QA, not your main translation engine**. [14]

LLM post-editing is where the whole pipeline becomes class-ready. OpenAI’s API supports the Responses API for text generation, structured outputs for schema-safe JSON, and Batch for asynchronous high-volume jobs at a **50% discount**. Current low-cost model pricing means a rewrite step can be extremely cheap at your scale. If you prefer local control, Qwen2.5-7B-Instruct is Apache 2.0, multilingual, and can run behind vLLM with an OpenAI-compatible API surface. The right role for an LLM here is not “translate from scratch.” It is “take raw MT + lexical evidence + POS + CEFR and emit a short, learner-friendly final `definition_zh` and `translation_zh`.” Recommended usage pattern: **cheap MT first, structured LLM post-edit second, human spot-check third**. [20]

Recommended hybrid workflows

The strongest workflow for your pilot is **Azure Translator F0 + lexical validation + lightweight post-editing**. Use Azure for raw `translation_zh` and for a first-pass translated gloss; then pass only the Chinese gloss through a rule-based compressor or very cheap LLM step to keep it short and learner-friendly. Cross-check nouns, verbs, and very common A1/A2 words against CC-CEDICT or Wiktionary candidates. For a pilot-sized set, this gives you near-zero direct translation cost, no display-attribution requirement, and a much cleaner final product than raw MT alone. At **1,000 translation units** or **10,000 translation units**, assuming roughly **100 English source characters per unit**, Azure F0 still covers the load at **$0** because the free tier is **2 million characters/month**. [21]

A close second is **Google Cloud Translation + post-editing**. This is especially good if you want glossaries or already operate on GCP. The key product trick is to avoid publishing raw Google output directly. Once you post-edit, Google’s FAQ says attribution is no longer required. For a small pilot, the **500,000 free characters/month** likely cover most needs anyway. I would choose this over Azure only if your team already has Google Cloud in place or if glossary control matters immediately. [22]

The best all-open, lowest-cash workflow is **LibreTranslate or Argos Translate + CC-CEDICT/Wiktionary + LLM post-editing**. If you already own the machine, the marginal translation cost is essentially zero. The trade-off is quality and engineering time. This workflow makes sense if privacy, offline operation, or vendor independence matters more than time-to-polish. For learner-facing Chinese, I would only ship this after a post-editing layer or strong human QA. [23]

For the LLM part, the economics are usually negligible at your scale. If you use **OpenAI Batch** with a low-cost model such as **GPT-5 nano**, and you budget roughly **120 input tokens + 40 output tokens** per row for a structured rewrite, then **1,000 post-edits** cost only a few cents and **10,000 post-edits** cost only a few tenths of a dollar at current published Batch rates. Even if your prompts are larger, the post-editing layer is still likely to be cheaper than human cleanup on every row. [24]

Implementation details and sample Python

A practical rule for your schema is this:

- Use an MT provider for `translation_zh`.
- Use **MT + lexicon evidence + rewrite** for `definition_zh`.
- Store **raw candidate text**, **final text**, **source/provider**, **license note**, and **QA status** separately.

A minimal `fetch_candidates.py` can pull raw MT plus lexical candidates:

```
# fetch_candidates.py
import csv
import json
import requests

AZURE_KEY = "YOUR_KEY"
AZURE_REGION = "YOUR_REGION"
AZURE_URL = (
    "https://api.cognitive.microsofttranslator.com/"
    "translate?api-version=3.0&from=en&to=zh-Hans"
)

def azure_translate_many(texts: list[str]) -> list[str]:
    headers = {
        "Ocp-Apim-Subscription-Key": AZURE_KEY,
        "Ocp-Apim-Subscription-Region": AZURE_REGION,
        "Content-Type": "application/json; charset=UTF-8",
    }
    body = [{"text": t} for t in texts]
    r = requests.post(AZURE_URL, headers=headers, json=body, timeout=30)
    r.raise_for_status()
    data = r.json()
    return [item["translations"][0]["text"] for item in data]

with open("level_001_seed.csv", newline="", encoding="utf-8") as f:
    rows = list(csv.DictReader(f))

definition_inputs = [r["definition_en"] for r in rows]
sentence_inputs = [r["sentence_en"] for r in rows]

definition_mt = azure_translate_many(definition_inputs)
sentence_mt = azure_translate_many(sentence_inputs)

packages = []
for row, def_zh_raw, sent_zh_raw in zip(rows, definition_mt, sentence_mt):
    packages.append({
```

```
        "headword": row["headword"],
        "pos": row["pos"],
        "definition_en": row["definition_en"],
        "sentence_en": row["sentence_en"],
        "definition_zh_raw": def_zh_raw,
        "translation_zh_raw": sent_zh_raw,
        "definition_zh_source": "azure_translator",
        "translation_zh_source": "azure_translator",
        "license_note":
"Azure Translator; no attribution required per Microsoft FAQ",
        "qa_status": "draft"
    })

with open("level_001_packages.raw.json", "w", encoding="utf-8") as f:
    json.dump(packages, f, ensure_ascii=False, indent=2)
```

That request shape matches Microsoft's documented REST API: `POST` to the translator endpoint with a JSON array body, and `to=zh-Hans` or `to=zh-Hant` in the query string. [25]

If you want a Google Cloud version for small jobs, Basic v2 is also simple:

```
# google_basic_translate.py
import requests

API_KEY = "YOUR_GOOGLE_API_KEY"
url = f"https://translation.googleapis.com/language/translate/v2?key={API_KEY}"

payload = {
    "q": [
        "the physical or mental power to do something",
        "She has the ability to learn quickly."
    ],
    "target": "zh-CN"
}

r = requests.post(url, json=payload, timeout=30)
r.raise_for_status()
print(r.json())
```

Google documents the v2 JSON request/response shape and lists Simplified Chinese as `zh-CN` or `zh`, with Traditional Chinese as `zh-TW`. Google also recommends keeping requests small for lower latency. [26]

If you need a fully open-source HTTP fallback:

```python
# libretranslate_example.py
import requests

url = "https://libretranslate.com/translate"
payload = {
    "q": [
        "the physical or mental power to do something",
        "She has the ability to learn quickly."
    ],
    "source": "en",
    "target": "zh",
    "format": "text",
    "api_key": "YOUR_KEY_IF_REQUIRED"
}

r = requests.post(url, json=payload, timeout=30)
r.raise_for_status()
print(r.json())
```

LibreTranslate's docs show that `q` can be a string or an array, that self-hosted instances do not require `api_key`, and that the service may return `429` when you need to slow down. [27]

A minimal `rewrite_and_draft.py` should then normalize the raw translations into learner-friendly Chinese. Even without an LLM, you can start with a rule-based pass:

```python
# rewrite_and_draft.py
import json

def compress_definition_zh(raw: str) -> str:
    text = raw.strip()
    replacements = {
        "做某事的身体或心理力量": "能力",
        "做某事的身体或精神力量": "能力",
    }
    return replacements.get(text, text)

with open("level_001_packages.raw.json", encoding="utf-8") as f:
    packages = json.load(f)

for pkg in packages:
    pkg["definition_zh"] = compress_definition_zh(pkg["definition_zh_raw"])
    pkg["translation_zh"] = pkg["translation_zh_raw"]
    pkg["qa_status"] = "needs_review"
```

```
with open("level_001_packages.draft.json", "w", encoding="utf-8") as f:
    json.dump(packages, f, ensure_ascii=False, indent=2)
```

For higher quality, replace that rule layer with a cheap structured LLM rewrite that takes `definition_en`, `definition_zh_raw`, `translation_zh_raw`, `headword`, `pos`, and `cefr_level`, then returns JSON with `definition_zh`, `translation_zh`, and a `confidence_note`. OpenAI's Responses API and Structured Outputs are built for exactly this kind of schema-safe normalization, and OpenAI Batch is the cheapest way to do it at scale. Local Qwen2.5 behind vLLM gives you the same architectural pattern if you prefer local inference. [28]

For Supabase CSV export, keep the translation audit metadata outside or alongside the teaching fields instead of overwriting it. A good flat export shape is:

```
headword,pos,cefr_level,definition_en,definition_zh,translation_zh,
definition_zh_raw,translation_zh_raw,
definition_zh_source,translation_zh_source,
license_note,qa_status
```

That gives you one clean row for the app and one complete row for QA/legal review. If you later decide to switch providers, you can re-run only the raw columns without losing human edits. This is especially useful if you mix vendor MT with CC resources, because you can keep share-alike sources confined to the audit trail and final normalized text confined to your production columns. This is the cleanest way to preserve provenance while still shipping simple Supabase rows. [29]

Minimal implementation plan

For a **Level 1 pilot of 20–80 words**, I would implement the smallest workable stack:

1. **Pick one primary MT provider.** Use **Azure Translator F0** unless you already live on GCP. It is the best free-tier/citation-safety tradeoff I found. [30]

2. **Add an audit-ready raw layer.** In `fetch_candidates.py`, store `definition_zh_raw`, `translation_zh_raw`, provider name, timestamp, and any license/attribution note. If you consult CC-CEDICT or Wiktionary, store them as `candidate_zh_1`, `candidate_zh_2`, not as final copy. [31]

3. **Normalize only the fields that need it.** Keep `translation_zh` close to the MT output unless it is obviously awkward. Spend your polishing budget on `definition_zh`, because that is the field that most needs classroom-friendly wording. [32]

4. **Human-check a small slice before scaling.** Review the first 20 rows manually and look for recurring problems: over-literal glosses, wrong POS, overlong Chinese, or sentence translations that do not sound natural. Then encode those fixes into rules or an LLM prompt. [33]

5. **Export two artifacts, not one.** Generate a clean Supabase CSV for production fields and a richer audit CSV/JSON for provenance. This is the simplest way to stay flexible if you later switch providers or need to answer a licensing question. [34]

6. **Only after the pilot, scale to the full Oxford slice.** At that point, decide whether `definition_zh` can be rule-normalized cheaply, or whether it deserves a batched LLM post-edit pass. OpenAI Batch or a local Qwen2.5 service both fit that second-stage role well. [35]

Open questions and limitations

Two source-specific issues are worth flagging before you lock your production design. DeepL's public API documentation is currently in a plan-transition state, with older `API Free` / `API Pro` language still visible in some places and newer `API Developer` / `API Growth` terms in others, so I would verify the exact current commercial plan and data-handling terms at signup time. [36]

CC-CEDICT's own official pages are inconsistent about the current share-alike version, with one page referencing **CC BY-SA 3.0** and another referencing **CC BY-SA 4.0**. That does not block use, but it does mean your audit log should capture the exact source URL and date whenever you ingest CC-CEDICT-derived candidates. [13]

Finally, the quality labels in this report are implementation judgments based on feature sets, licensing, JSON/API ergonomics, and operational fit for your Level 1 app. They are not formal head-to-head MT benchmark scores. For your actual content, the decisive test is a 20-row pilot with your real definitions and example sentences. [37]

---

[1] [2] [8] [16] [21] [30] Service limits - Translator - Foundry Tools | Microsoft Learn
https://learn.microsoft.com/en-us/azure/ai-services/translator/service-limits

[3] [11] [19] [23] Overview | LibreTranslate
https://docs.libretranslate.com/api/

[4] [13] CC-CEDICT Home [CC-CEDICT WIKI]
https://cc-cedict.org/wiki/

[5] [32] Translate Text - DeepL Documentation
https://developers.deepl.com/api-reference/translate?utm_source=chatgpt.com

[6] [14] Wiktionary:Copyrights
https://en.wiktionary.org/wiki/Wiktionary%3ACopyrights?utm_source=chatgpt.com

[7] [25] [37] Translator Translate Method - Foundry Tools | Microsoft Learn
https://learn.microsoft.com/en-us/azure/ai-services/translator/text-translation/reference/v3/translate

[9] [17] [22] Cloud Translation
https://cloud.google.com/translate?utm_source=chatgpt.com

[10] [18] [36] DeepL API plans
https://support.deepl.com/hc/en-us/articles/360021200939-DeepL-API-plans?utm_source=chatgpt.com

12 GitHub - argosopentech/argos-translate: Open-source offline translation library written in Python · GitHub
https://github.com/argosopentech/argos-translate/

15 GPT-5 mini Model | OpenAI API
https://developers.openai.com/api/docs/models/gpt-5-mini?utm_source=chatgpt.com

20 28 Text generation | OpenAI API
https://developers.openai.com/api/docs/guides/text?utm_source=chatgpt.com

24 GPT-5 nano Model | OpenAI API
https://developers.openai.com/api/docs/models/gpt-5-nano?utm_source=chatgpt.com

26 Translate text with Cloud Translation | Google Cloud Documentation
https://docs.cloud.google.com/translate/docs/translate-text

27 API Usage | LibreTranslate
https://docs.libretranslate.com/guides/api_usage/

29 34 Attribution Requirements | Cloud Translation
https://docs.cloud.google.com/translate/attribution?utm_source=chatgpt.com

31 CC-CEDICT Editor
https://cc-cedict.org/editor/editor.php?handler=Download

33 Evaluate translation models
https://docs.cloud.google.com/translate/docs/advanced/translation-model-evaluation?utm_source=chatgpt.com

35 Batch API
https://developers.openai.com/api/docs/guides/batch?utm_source=chatgpt.com